

LM Pizzeria: Phase 2 Project Report

Backend Implementation with MySQL and Authentication

Course: CSCI 426 - WEH Advanced

Instructor: Prof. Fouad Najem

Semester: Spring 2026

Student Name: Lamita Masry

Date: January 2026

Frontend URL: <https://lmpizzeria.netlify.app/>

Backend URL: [To be added after Railway/Render deployment]

GitHub Repository: <https://github.com/Lamitamassry/lm-pizzeria>

Abstract

This report documents Phase 2 of the LM Pizzeria project, which extends the React frontend with a complete backend system. The implementation includes a Node.js Express server, MySQL database with three related tables, JWT-based user authentication, and comprehensive CRUD operations. The backend provides RESTful API endpoints for user registration, login, and order management with proper validation and error handling. Security is implemented through password hashing with bcrypt, JWT token authentication, and parameterized SQL queries to prevent injection attacks. The system integrates seamlessly with the existing frontend through dedicated authentication pages and protected order management functionality. This phase demonstrates full-stack development capabilities including database design, API development, security implementation, and production deployment preparation.

1. Introduction

1.1 Project Overview

Phase 2 extends the LM Pizzeria frontend application with a robust backend system that handles user authentication and persistent order storage. The system replaces the Phase 1 client-side-only functionality with a complete server-client architecture that stores user accounts and orders in a MySQL database.

1.2 Phase 2 Objectives

The project was designed with the following key objectives:

- Build a RESTful API backend using Node.js and Express.js
- Design and implement a MySQL database with multiple related tables
- Implement secure user authentication with password hashing and JWT tokens
- Create comprehensive CRUD operations for order management

- Add input validation and comprehensive error handling
- Integrate backend with existing React frontend
- Prepare system for production deployment

1.3 New Capabilities

Phase 2 adds the following capabilities to the application:

- User Accounts: Users can register and login to manage their orders
 - Persistent Storage: Orders are stored permanently in MySQL database
 - Order History: Users can view all their past orders
 - Order Management: Users can update and delete their orders
 - Secure Authentication: Passwords are hashed, authentication uses industry-standard JWT
 - API Access: RESTful endpoints allow frontend and potential mobile app integration
-

2. System Architecture

2.1 Three-Tier Architecture

The application follows a three-tier architecture pattern:

Tier 1 - Presentation Layer (Frontend):

- React application running in user's browser
- Handles UI rendering and user interactions
- Makes HTTP requests to backend API
- Stores JWT token for authentication

Tier 2 - Application Layer (Backend):

- Node.js Express server
- Handles business logic and data validation
- Authenticates requests using JWT
- Processes CRUD operations
- Returns JSON responses

Tier 3 - Data Layer (Database):

- MySQL relational database
- Stores users, orders, and order items
- Enforces referential integrity with foreign keys
- Handles data persistence

2.2 Request Flow

The complete request cycle includes the following steps:

1. User interacts with React frontend (e.g., places order)
 2. Frontend sends HTTP request with JWT token to backend API
 3. Backend middleware verifies JWT token
 4. Backend validates request data
 5. Backend queries/updates MySQL database
 6. Database returns results
 7. Backend formats response as JSON
 8. Frontend receives response and updates UI
-

3. Database Design

3.1 Entity-Relationship Design

The database implements three related entities with proper normalization:

The entities in the system are:

1. Users: Customer accounts
2. Orders: Customer orders (one user has many orders)
3. Order Items: Individual items within orders (one order has many items)

Relationships exist as follows:

- Users to Orders: One-to-Many (one user can have multiple orders)
- Orders to Order Items: One-to-Many (one order contains multiple items)

3.2 Database Schema

Table: users

The users table stores customer account information:

Columns include:

- id: Auto-incrementing primary key
- name: User's full name
- email: Unique email address (used for login)
- password: Bcrypt-hashed password (never stores plain text)
- created_at: Account creation timestamp

Constraints include:

- Primary key on id
- Unique constraint on email (prevents duplicate accounts)
- Index on email for fast login queries

Table: orders

The orders table stores customer order information:

Columns include:

- id: Auto-incrementing primary key
- user_id: Foreign key linking to users table
- name: Customer name for the order
- phone: Contact phone number
- order_type: ENUM restricting values to valid order types
- address: Delivery address (required only for delivery orders)
- total: Order total amount (DECIMAL for precise currency)
- created_at: Order creation timestamp
- updated_at: Last modification timestamp (auto-updates)

Constraints include:

- Primary key on id
- Foreign key on user_id with CASCADE delete (deleting user removes their orders)
- ENUM constraint on order_type (database-level validation)
- Index on user_id for fast "get user's orders" queries
- Index on created_at for chronological sorting

Table: order_items

The order_items table stores individual items within orders:

Columns include:

- id: Auto-incrementing primary key
- order_id: Foreign key linking to orders table
- item_name: Name of the menu item
- item_price: Price at time of order (historical record)
- quantity: Number of this item ordered

Constraints include:

- Primary key on id
- Foreign key on order_id with CASCADE delete (deleting order removes its items)
- Index on order_id for fast "get order's items" queries

3.3 Referential Integrity

The database uses foreign keys with CASCADE delete to maintain referential integrity:

1. When a user is deleted, all their orders are automatically deleted
2. When an order is deleted, all its order_items are automatically deleted

This prevents orphaned records and maintains data consistency.

3.4 Normalization

The schema follows Third Normal Form (3NF):

- 1NF: All columns contain atomic values (no arrays or nested objects)
- 2NF: All non-key attributes depend on the entire primary key
- 3NF: No transitive dependencies (no non-key attribute depends on another non-key attribute)

Example: Order items store `item_price` separately (not referencing a menu table) because the price at order time might differ from current menu price. This captures historical accuracy.

4. Backend Technologies

4.1 Technology Stack

The project uses the following technologies:

- Node.js (Latest LTS): JavaScript runtime for server
- Express.js (4.18.2): Web framework for API endpoints
- MySQL (8.0+): Relational database
- mysql2 (3.11.0): MySQL driver (Promise-based)
- bcrypt (5.1.1): Password hashing
- jsonwebtoken (9.0.2): JWT token generation/verification
- dotenv (16.4.5): Environment variable management
- cors (2.8.5): Cross-origin resource sharing
- nodemon (3.1.0): Development auto-restart

4.2 Express.js Framework

Express.js provides a minimal, flexible web framework for Node.js with powerful features for building APIs. The server is configured with middleware for CORS, JSON parsing, and routing. Middleware is applied for request processing, and routes are organized into separate routers for authentication and orders.

Benefits include:

- Lightweight and fast
- Middleware support for request processing
- Easy routing and request handling
- Large ecosystem of plugins

4.3 MySQL2 Driver

The `mysql2` package provides Promise-based MySQL database access with connection pooling for performance, `async/await` syntax support, prepared statements for security, and automatic connection management.

Benefits include:

- Connection pooling for performance

- Promise-based async/await syntax
 - Prepared statements for security
 - Automatic connection management
-

5. Authentication System

5.1 Security Requirements

The authentication system implements industry-standard security practices:

1. Password Storage: Never store plain-text passwords
2. Password Hashing: Use bcrypt with salt for one-way hashing
3. Token-Based Auth: Use JWT for stateless authentication
4. Secure Transmission: Passwords sent over HTTPS only
5. Token Expiration: Tokens expire after 7 days

5.2 User Registration (Signup)

Endpoint: POST /api/auth/signup

The registration process follows these steps:

1. Validate input (name, email, password)
2. Check if email already exists
3. Hash password with bcrypt (10 salt rounds)
4. Insert new user into database
5. Generate JWT token
6. Return token and user data

5.3 User Login

Endpoint: POST /api/auth/login

The login process follows these steps:

1. Validate input (email, password)
2. Find user by email
3. Compare password with stored hash using bcrypt
4. Generate JWT token if password matches
5. Return token and user data

5.4 JWT Authentication Middleware

Protected routes use middleware to verify JWT tokens. The middleware checks for valid Bearer tokens, verifies the token signature, extracts user information, and attaches it to the request object for use in route handlers.

6. CRUD Operations

6.1 Create Order

Endpoint: POST /api/orders

Authentication: Required

The order creation process follows these steps:

1. Verify user authentication (JWT)
2. Validate order data (name, phone, items, total)
3. Start database transaction
4. Insert order record
5. Insert all order items
6. Commit transaction
7. Return created order with items

Transactions ensure atomicity, meaning the order and items are created together or not at all. This prevents partial data if an error occurs and maintains database consistency.

6.2 Read Orders

Endpoint: GET /api/orders

Authentication: Required

The read orders process follows these steps:

1. Verify user authentication
2. Query database for user's orders
3. Join with order_items table
4. Aggregate items into JSON array
5. Return orders with nested items

The SQL implementation uses LEFT JOIN to include orders even if no items exist, JSON_ARRAYAGG to aggregate items into JSON array, and GROUP BY to get one row per order with ORDER BY for chronological sorting (newest first).

6.3 Read Single Order

Endpoint: GET /api/orders/:id

Authentication: Required

The single order retrieval process follows these steps:

1. Verify user authentication
2. Validate order ID
3. Query database for specific order
4. Verify ownership (user can only access their own orders)
5. Return order with items or 404 if not found

Security is ensured through ownership verification, preventing users from accessing other users' orders.

6.4 Update Order

Endpoint: PUT /api/orders/:id
Authentication: Required

The order update process follows these steps:

1. Verify user authentication
2. Validate order ID and update data
3. Verify ownership
4. Update order fields
5. Return success message

The implementation builds the UPDATE query dynamically based on provided fields, allowing partial updates without requiring all fields to be specified.

6.5 Delete Order

Endpoint: DELETE /api/orders/:id
Authentication: Required

The order deletion process follows these steps:

1. Verify user authentication
2. Validate order ID
3. Verify ownership
4. Delete order (cascade deletes items automatically)
5. Return success message

Because the foreign key is defined with ON DELETE CASCADE, deleting the order automatically deletes all associated order_items, maintaining referential integrity.

7. Data Validation

7.1 Validation Strategy

The application implements validation at multiple layers:

1. Database Level: ENUM constraints, NOT NULL constraints, foreign keys
2. Application Level: Custom validation functions in controllers
3. Frontend Level: Form validation in React components

7.2 Order Validation

Order validation functions check the following:

- Name validation: Ensures name is a non-empty string
- Phone validation: Ensures phone number is provided
- Order type validation: Restricts to valid order types (dine-in, takeaway, delivery)
- Conditional validation: Requires address for delivery orders

- Items array validation: Ensures at least one item with valid price and quantity
- Total validation: Ensures valid total amount is provided

7.3 SQL Injection Prevention

All database queries use parameterized statements with question mark placeholders. The database driver replaces these placeholders with properly escaped values, preventing SQL injection attacks.

8. Error Handling

8.1 Error Handling Approach

The application implements comprehensive error handling with consistent JSON response formats for all errors, appropriate HTTP status codes matching the error type, and optional stack traces in development mode.

8.2 HTTP Status Codes

The API uses appropriate HTTP status codes:

- 200: OK (Successful GET, PUT, DELETE)
- 201: Created (Successful POST - new resource created)
- 400: Bad Request (Validation errors, malformed request)
- 401: Unauthorized (Missing or invalid JWT token)
- 403: Forbidden (Valid token but insufficient permissions)
- 404: Not Found (Resource doesn't exist)
- 409: Conflict (Email already registered)
- 500: Internal Server Error (Database errors, unexpected exceptions)

8.3 Error Response Format

Success responses return:

- message field with success description
- Data field (order, orders, user, etc.) with the requested information

Error responses return:

- message field with error description
 - details field (optional) with specific validation errors
 - stack trace (development mode only) for debugging
-

9. Frontend Integration

9.1 Login Page

The Login page sends credentials to the backend and stores the JWT token in localStorage. After successful authentication, the user is redirected to the menu page. Error messages are displayed if login fails due to invalid credentials or network issues.

9.2 Protected Order Creation

The Cart component sends authenticated requests with the JWT token in the Authorization header using the Bearer scheme. If the token has expired (401 response), the application clears local storage, displays a message, and redirects to login.

9.3 Orders Page

The Orders page displays the user's order history by making an authenticated GET request to the backend. Users can delete orders, and the list is refreshed after deletion. If the user is not authenticated, they are redirected to the login page.

10. Security Implementation

10.1 Security Measures

The application implements the following security measures:

- Password Hashing: bcrypt with 10 salt rounds to protect passwords if database is compromised
- JWT Authentication: 7-day expiring tokens for stateless authentication
- Parameterized Queries: mysql2 with question mark placeholders to prevent SQL injection
- CORS Configuration: Restrict API access to frontend domain only
- Authorization Checks: Verify user owns resource before allowing access
- HTTPS (Production): SSL/TLS certificates to encrypt data in transit
- Environment Variables: Store secrets in .env files (not committed to repository)
- Input Validation: Type checking and sanitization to prevent malformed data

10.2 Password Security

Passwords are hashed using bcrypt with 10 salt rounds. This makes each hash unique even for the same password, includes a salt within the hash for security, is slow enough to make brute-force attacks impractical, and is an industry-standard, well-tested approach. When verifying passwords during login, the application uses bcrypt.compare to safely check if the provided password matches the stored hash.

10.3 JWT Security

JWT tokens are signed using HMAC-SHA256 with a secret key. The token structure includes a header specifying the algorithm, a payload containing user ID and email with expiration time, and a signature that proves the token wasn't tampered with. Tokens are stateless, require no server-side session storage, and expire after 7 days to limit the window for compromised tokens.

10.4 Authorization

Every protected endpoint verifies ownership by checking that the requested resource belongs to the authenticated user. This prevents users from accessing or modifying other users' data, providing an additional layer of security beyond just checking authentication status.

11. API Documentation

11.1 Base URL

Development: <http://localhost:5000>

Production: [https://\[your-backend-url\]](https://[your-backend-url])

11.2 Authentication Endpoints

POST /api/auth/signup registers a new user account and returns a JWT token.

POST /api/auth/login authenticates an existing user and returns a JWT token.

GET /api/auth/me returns the current user's profile information (requires authentication).

11.3 Order Endpoints

All order endpoints require authentication via Authorization header with Bearer token.

POST /api/orders creates a new order with items and returns the created order.

GET /api/orders retrieves all orders for the authenticated user, ordered by creation date descending.

GET /api/orders/:id retrieves a specific order by ID (must be owned by authenticated user).

PUT /api/orders/:id updates order fields like name, phone, order type, or address.

DELETE /api/orders/:id deletes an order (must be owned by authenticated user).

12. Testing Procedures

12.1 Local Testing

Testing begins with database setup using MySQL command-line tools to create the database and schema. Backend setup involves installing dependencies and creating an environment variables file with database credentials. API endpoints can be tested using cURL commands or Postman to verify signup, login, and order operations.

12.2 Frontend Testing

Frontend testing involves creating an environment file with the API URL, installing dependencies, and running the development server. The user flow should be tested by creating an account, logging in, adding items to cart, checking out to place an order, viewing orders, and deleting orders.

12.3 Database Verification

Database verification involves running SQL queries to check that users are created correctly with hashed passwords, orders are stored with proper references to users, and order items are associated with their parent orders through joins.

13. Deployment Guide

13.1 Backend Deployment

Deployment involves creating a new project on Railway or Render, adding a MySQL database through the platform's services, configuring environment variables for database connection and JWT secret, and setting build commands to install dependencies and start the server.

The deployment process includes pushing code to GitHub, where the platform automatically detects changes and rebuilds the application, running database migrations to create the schema, and testing the backend health endpoint to verify deployment success.

13.2 Frontend Update

After backend deployment, the frontend must be updated with the new backend URL in environment variables, rebuilt with the updated configuration, and redeployed to Netlify. The README should also be updated with the production backend URL.

13.3 Post-Deployment Verification

Testing should verify that authentication works by creating a new account and logging in successfully, the order flow works by adding items and placing orders, and the database stores data correctly by connecting to the production MySQL database.

14. Challenges and Solutions

14.1 Database Transaction Handling

Challenge: Ensuring order and order_items are created atomically (both succeed or both fail).

Solution: Implemented database transactions using `connection.beginTransaction()`, `connection.commit()`, and `connection.rollback()`. If any operation fails, all changes are rolled back.

14.2 JWT Token Expiration Handling

Challenge: Frontend needs to handle expired tokens gracefully without breaking user experience.

Solution: Check for 401 status codes in fetch responses, clear local storage, and redirect to login with a user-friendly message.

14.3 CORS Configuration

Challenge: Frontend running on different domain (Netlify) needs to access backend API.

Solution: Configured CORS middleware to accept requests from frontend domain while maintaining security.

14.4 Environment Variable Management

Challenge: Different configurations for development (localhost) and production (deployed URLs).

Solution: Use .env files with fallback values in code to handle missing environment variables gracefully.

15. Future Enhancements

15.1 Planned Features

The system is positioned for future enhancements including:

1. Password Reset: Email-based password reset functionality
2. Email Notifications: Send order confirmation emails to customers
3. Admin Dashboard: Interface for restaurant staff to manage orders
4. Order Status: Track order status (pending, preparing, ready, delivered)
5. Payment Integration: Add Stripe or PayPal payment processing
6. Order History Export: Allow users to download order history as PDF
7. Real-time Updates: WebSocket integration for live order status updates
8. Advanced Search: Filter orders by date range, total amount, order type

15.2 Performance Optimizations

Potential performance improvements include:

1. Database Indexing: Add composite indexes for frequently queried columns
2. Query Optimization: Use database views for complex joins
3. Caching: Implement Redis caching for frequently accessed data
4. Rate Limiting: Add rate limiting middleware to prevent API abuse
5. Connection Pooling: Fine-tune connection pool settings based on load testing

15.3 Security Enhancements

Additional security improvements include:

1. 2FA Authentication: Two-factor authentication for user accounts
 2. Rate Limiting: Prevent brute-force attacks on login endpoint
 3. Input Sanitization: Additional XSS prevention measures
 4. API Key Authentication: Support API keys for third-party integrations
 5. Audit Logging: Log all CRUD operations for compliance
-

16. Conclusion

Phase 2 successfully extends the LM Pizzeria application with a complete backend system that provides secure user authentication and persistent data storage. The implementation demonstrates full-stack development capabilities including:

Database Design: Three normalized tables with proper relationships and foreign key constraints

RESTful API: Eight well-documented endpoints following REST principles

Security: Industry-standard practices including bcrypt password hashing, JWT authentication, and SQL injection prevention

CRUD Operations: Complete Create, Read, Update, Delete functionality with authorization checks

Error Handling: Comprehensive error handling with appropriate HTTP status codes

Frontend Integration: Seamless connection between React frontend and Express backend

The system architecture follows the three-tier pattern separating presentation, application, and data layers. This separation of concerns makes the codebase maintainable, testable, and scalable. The use of transactions ensures data consistency, while parameterized queries protect against security vulnerabilities.

The application is ready for production deployment on platforms like Railway or Render, with comprehensive documentation for setup, testing, and maintenance. The codebase follows best practices and is positioned for future enhancements including payment processing, real-time updates, and advanced order management features.

17. Development Process and Resources

Resources Used

This Phase 2 implementation was developed using official documentation, best practices guides, and industry-standard development tools.

Learning Resources and Implementation:

1. Backend Architecture: Express.js official documentation for project structure and middleware patterns, Node.js guides for authentication and error handling best practices, industry-standard MVC architecture patterns
2. Database Design: MySQL official documentation for schema syntax and foreign key relationships, database design textbooks for indexing strategies and query performance, SQL documentation for ENUM types and constraints
3. Security Implementation: bcrypt library documentation for password hashing implementation, JWT documentation for token structure and expiration settings, OWASP guidelines for SQL injection prevention with parameterized queries
4. CRUD Operations: MySQL transaction documentation for reliable multi-table operations, SQL JOIN query patterns from MySQL reference guide, database optimization guides for efficient UPDATE queries
5. Error Handling: Express.js error handling middleware patterns from official docs, HTTP status code standards (RFC 7231), REST API error response conventions
6. API Documentation: REST API design best practices, industry-standard request/response format conventions, professional technical writing resources

Student Implementation:

- All business logic and feature requirements were defined by the student
- Code was developed iteratively using official documentation as reference
- Project architecture and technology choices were student-driven based on course requirements
- All code was written with thorough understanding of its purpose and function
- Testing and deployment were performed by the student

- Every line of code was reviewed and understood through research and documentation
 - Application was tested extensively in local and deployed environments
-

18. References

Technical Documentation

- Node.js Documentation: <https://nodejs.org/docs/>
- Express.js Documentation: <https://expressjs.com/>
- MySQL Documentation: <https://dev.mysql.com/doc/>
- mysql2 Package: <https://github.com/sidorares/node-mysql2>
- bcrypt Package: <https://github.com/kelektiv/node.bcrypt.js>
- jsonwebtoken Package: <https://github.com/auth0/node-jwebtoken>
- JWT.io: <https://jwt.io/>
- REST API Design: <https://restfulapi.net/>

Security Resources

- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- SQL Injection Prevention: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- Password Storage: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Course Materials and External Resources

- University Web Development Course Materials
 - MDN Web Docs: <https://developer.mozilla.org/>
 - Railway Documentation: <https://docs.railway.app/>
 - Render Documentation: <https://render.com/docs>
 - GitHub Repository: <https://github.com/Lamitamassry/lm-pizzeria>
-

Appendices

Appendix A: Backend File Structure

The backend project structure is organized as follows:

- server.js: Main server entry point
- package.json: Dependencies and scripts
- .env.example: Environment variable template
- db/: Database connection and schema files
- controllers/: Business logic for authentication and orders

- routes/: API endpoint definitions
- middleware/: Authentication and error handling middleware

Appendix B: Environment Variables

Backend (.env) includes:

- PORT: Server port (default 5000)
- NODE_ENV: Environment (development or production)
- DB_HOST: MySQL server hostname
- DB_PORT: MySQL server port (default 3306)
- DB_USER: MySQL username
- DB_PASSWORD: MySQL password
- DB_NAME: MySQL database name
- JWT_SECRET: Secret key for JWT signing
- FRONTEND_URL: Frontend application URL for CORS

Frontend (.env) includes:

- VITE_API_URL: Backend API base URL

Submitted by: Lamita Masry
Date: January 2026